



天津大学
Tianjin University

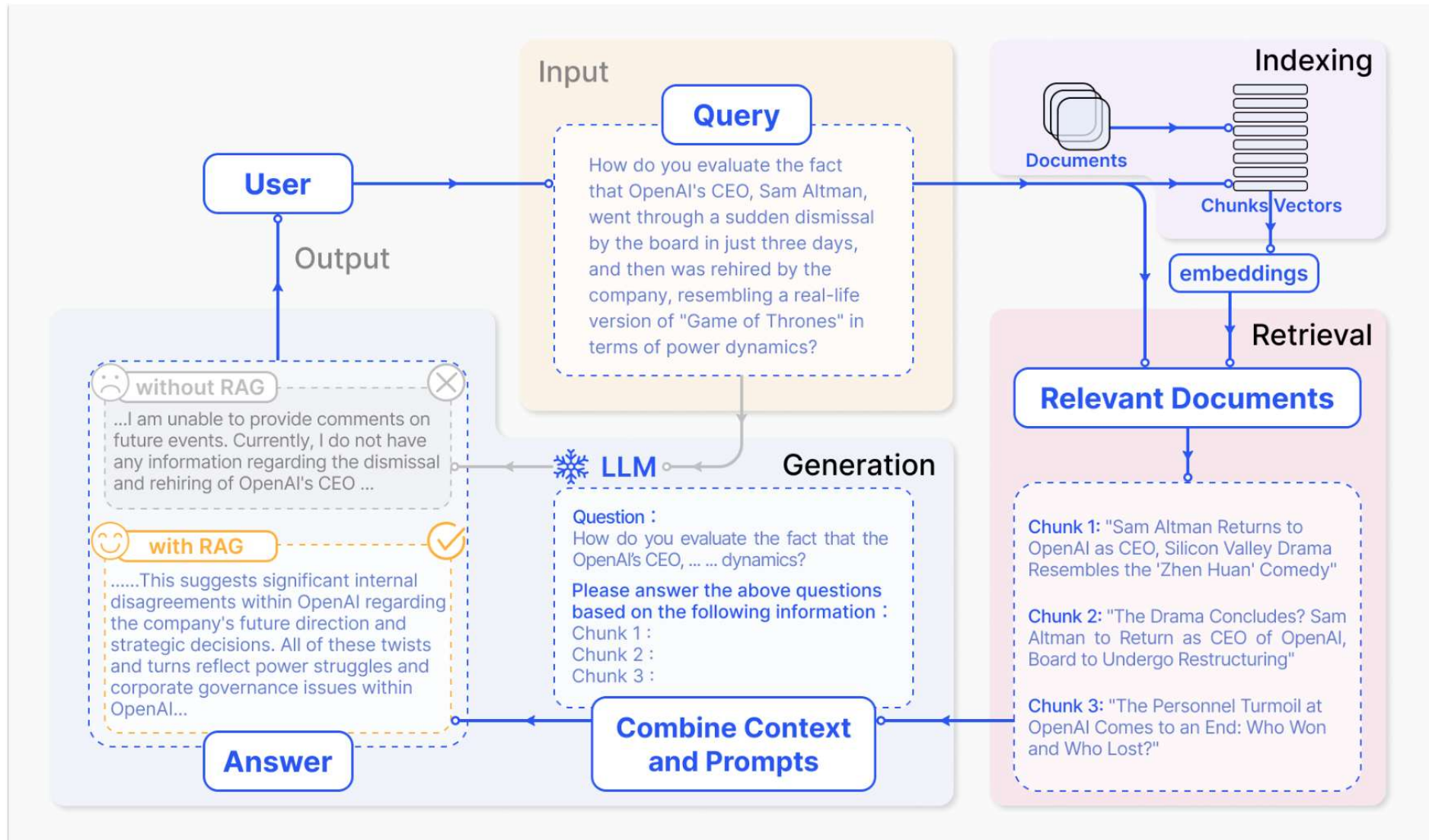


An Adaptive Vector Index Partitioning Scheme for Low-Latency RAG Pipeline

Junkyum Kim
jun-kyum.kim@gatech.edu
Georgia Institute of Technology

Divya Mahajan
divya.mahajan@gatech.edu
Georgia Institute of Technology

Background: RAG Pipeline



Background: Vector Search and IVF-PQ

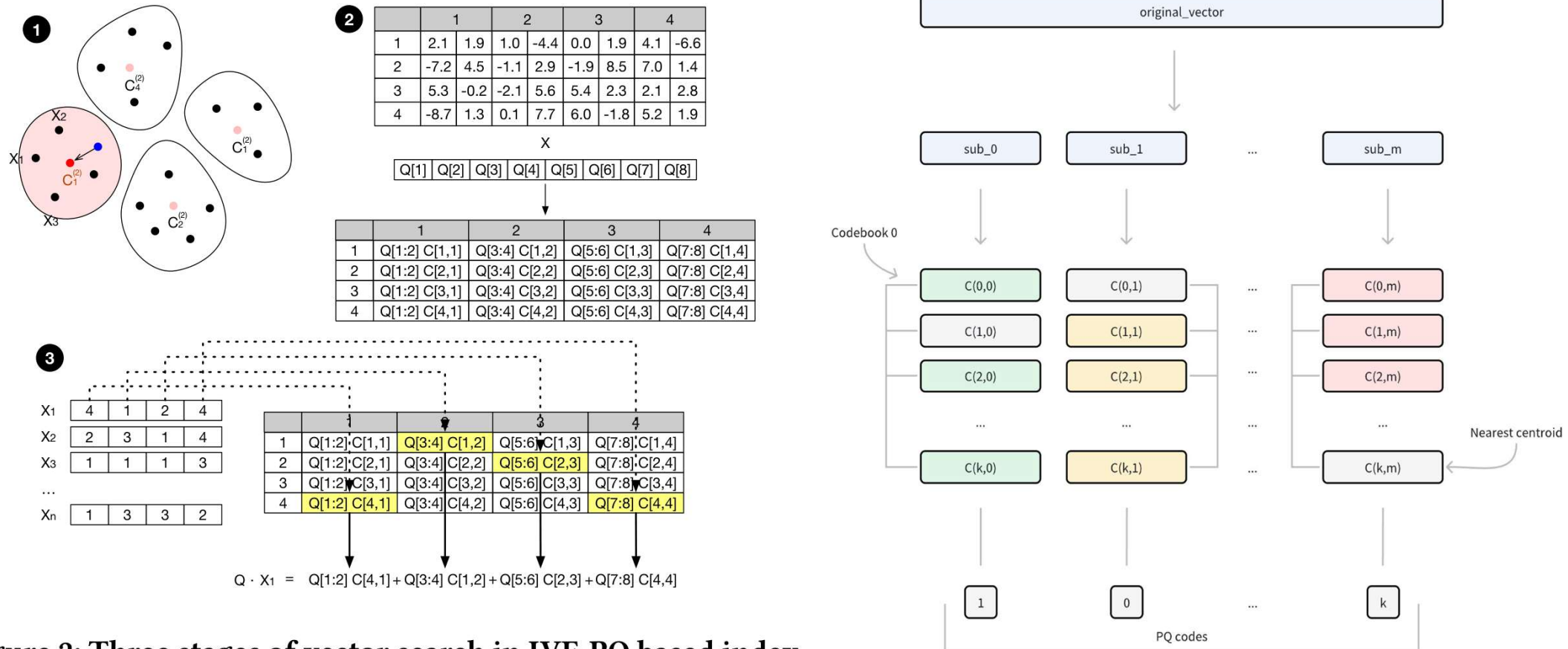


Figure 2: Three stages of vector search in IVF-PQ based index
- (1) Coarse quantization, (2) Building LUT, and (3) LUT scan and aggregation.

- k : Number of centroids per subspace, $k = 2^{n_{\text{bits}}}$.
- m : Number of sub-vectors the original vector is split into.
- n_{bits} : Bits used to encode each centroid's index.

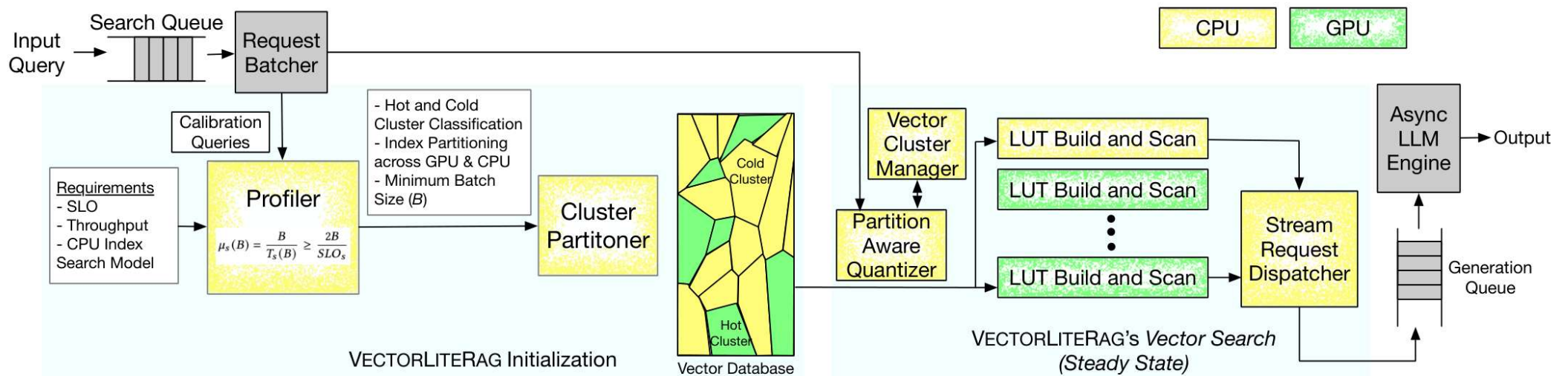
Background: Accelerate Vector Search

- **GPU based** approximate nearest neighbor search.
 - NVIDIA' s cuANN library and opti-mizations for frameworks like Faiss
- Specialized **hardware** optimizations for ANN search.
 - Rely on additional dedicated hardware solely for vector search operations.
- **System** level optimizations
 - DiskANN enhance scalability by leveraging hierarchical memory tiers to handle massive datasets

Not studied the impact on ANN search when integrated into a RAG pipeline during service time.

VectorLiteRAG: Adaptive Vector Index Partitioning

- Modeling CPU Index Search Time
- Estimating CPU Search Time
- Profile-guided hit rate estimation based on throughput estimation and SLO requirements.
- Converting hit rate to cluster classification and index partitioning.
- Re-calibrating the Hot and Cold Clusters



VectorLiteRAG: Adaptive Vector Index Partitioning

- Initialization

- Profiling a small number of requests to classify clusters, and determining the optimal index partitioning based on historical access patterns, latency statistics, and SLO requirements.

- Modeling CPU Index Search Time

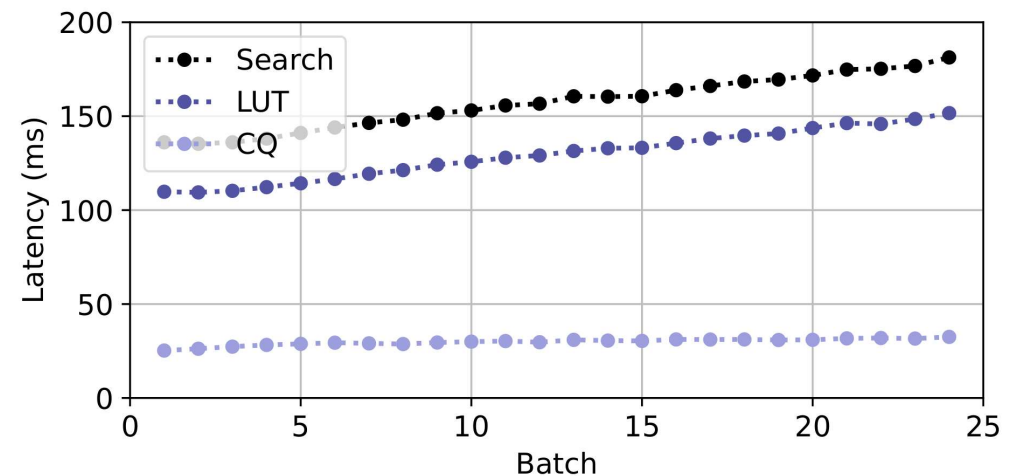
- Both LUT operations and coarse quantization exhibit an almost linear increase in execution time as batch size grows

$$T_{search}^{cpu} = T_{cq} + T_{lut} = (a_{cq}B + T_{c0}) + (a_{lut}B + T_{l0})$$

- If a query has a GPU hit rate of η , its CPU search time reduces to

$$T_{search}^{cpu} = T_{cq} + (1 - \eta)T_{lut}$$

The proportion of clusters retrieved from the GPU per request



- Estimating CPU Search Time

- Background

- Not all n_{probe} clusters are guaranteed to reside on the GPU.
 - As more clusters are accessed on the CPU, search time increases

- The **probability** that the query with the fewest GPU hits **in a batch has k hits**

$$P(k) = B \cdot BetaBin(k) [1 - F(x > k)]^{B-1}$$

- The **expected minimum hit rate**

$$E[\eta_{min}] = \frac{1}{n_{probe}} \sum_{k=0}^{n_{probe}} B \cdot x \cdot BetaBin(x) [1 - F(x)]^{B-1}$$

- The **CPU search time** and the **throughput**

$$T_{search}^{cpu} = T_{cq} + (1 - E[\eta_{min}])T_{lut} \quad \mu_{cpu} = \frac{B}{T_{search}^{cpu}}$$

VectorLiteRAG: Adaptive Vector Index Partitioning

- Profile-guided hit rate estimation based on throughput estimation and SLO requirements.
 - **Throughput Constraint:** ensure that search does not become a bottleneck in the pipeline

$$\mu_{cpu} \geq LLM_t$$

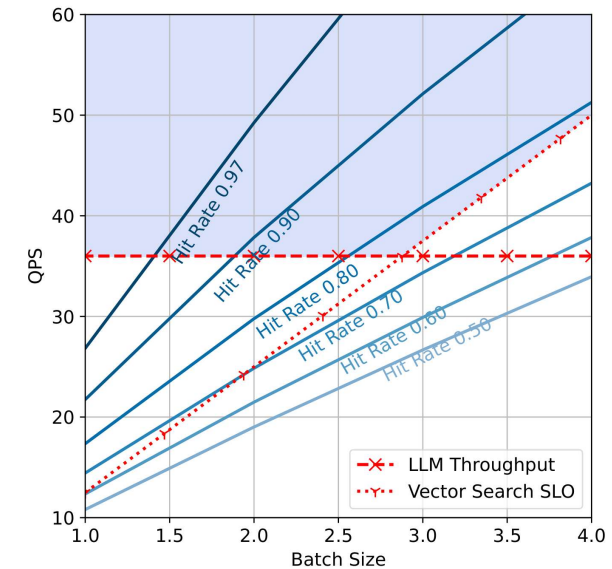
- **Latency Constraint:** The time to generate the first token must satisfy the SLO requirements.

$$SLO_{search}^{cpu} \geq \text{QueuingDelay} + T_{search}^{cpu}(B)$$

$$SLO_{search}^{cpu} \geq 2 \times T_{search}^{cpu}(B)$$

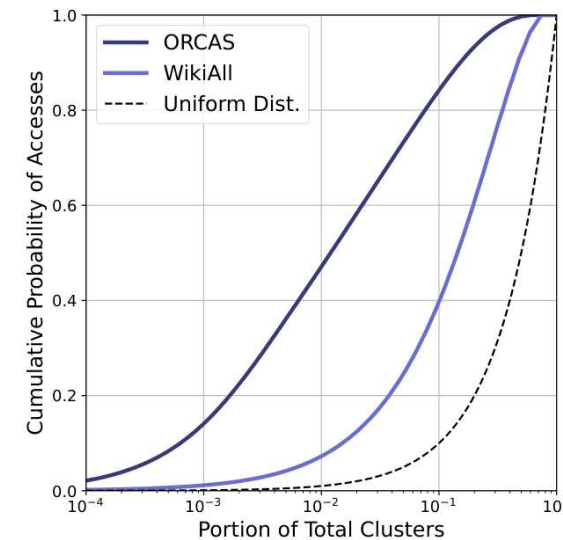
$$\Rightarrow \mu_{cpu} = \frac{B}{T_{search}^{cpu}} \geq \frac{2B}{SLO_{search}^{cpu}}$$

- The **smallest required hit rate** in the figure is **0.7**.



VectorLiteRAG: Adaptive Vector Index Partitioning

- Converting hit rate to cluster classification and index partitioning.
 - Analyze the **Cumulative Distribution Function** of these access patterns
 - Estimate the portion of clusters that should be **loaded onto the GPU** to achieve the desired hit rate.
 - The clusters are distributed across the GPUs **in a round-robin manner**.
- Re-calibrating the Hot and Cold Clusters
 - The partitioning of the vector index based on evolving hot and cold clusters must be **dynamically adjusted**.

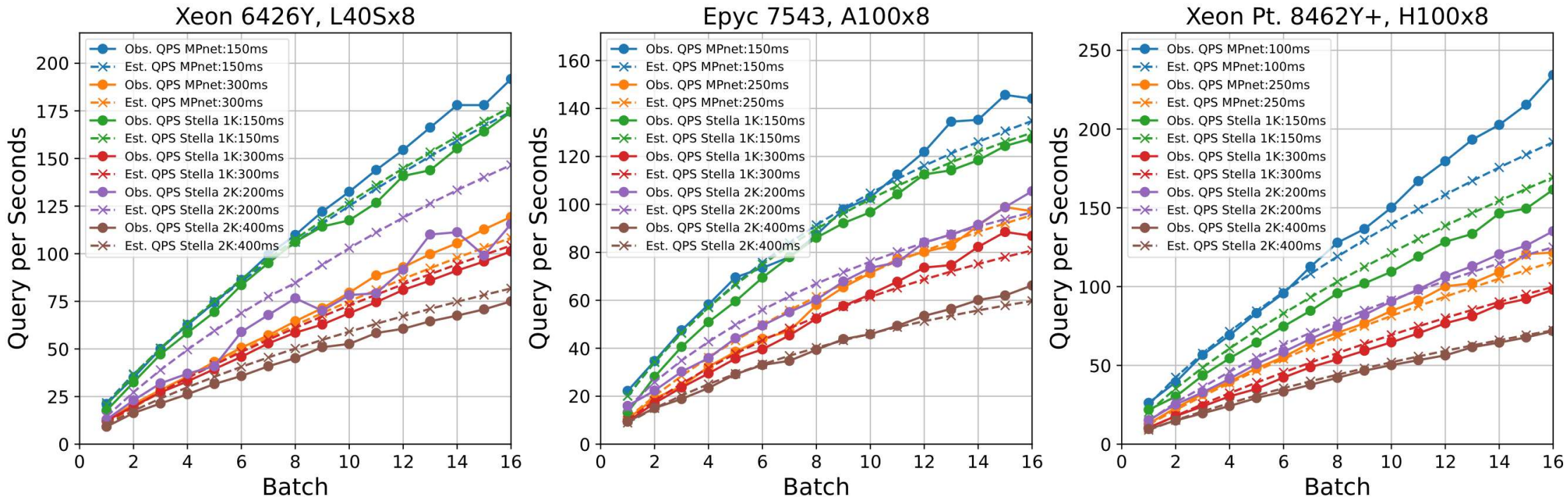


- Background
 - Some queries finish retrieval earlier and **remain idle** while waiting for slower queries to complete.
- Partitioning-aware Quantizer
 - It relies on a **centroid ID mapping table**, maintained by the index manager, to identify whether a cluster is **a hit or miss**
 - During coarse quantization, clusters are assigned based on their **hit or miss status**
- Streamlined Result Dispatcher
 - **Dynamically merging and dispatching** results to the LLM as soon as the top-k nearest neighbors are retrieved
 - If a query has completed scanning all the relevant clusters, and the GPU results are already merged, the dispatcher **immediately forwards the results** for document retrieval and LLM generation

Evaluation: Setup

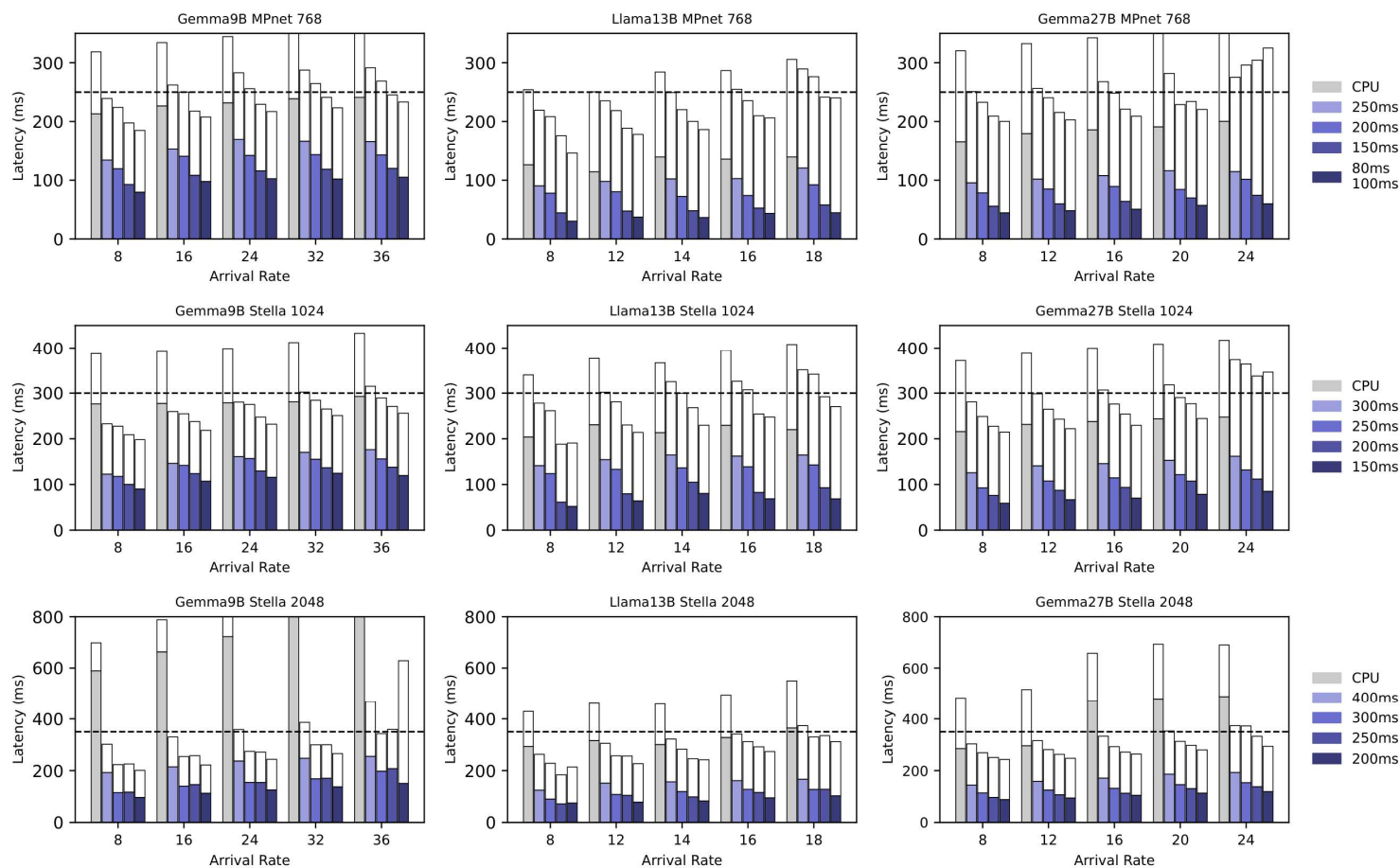
- **Hardware:** each equipped with eight NVIDIA GPUs
 - L40S GPUs with 48GB of GDDR memory + dual Xeon 6426Y CPUs.
 - A100 GPUs with 80GB of HBM + dual EPYC 7543 CPUs.
 - H100 GPUs with 80GB of HBM + Xeon Platinum 8462Y CPUs.
- **Models:** Gemma 9B, Llama 13B, Gemma 27B
 - vLLM as the LLM serving framework
- **Baselines:** the vanilla FAISS-CPU implementation
- **Datasets:** MPNet 768, Stella 1024, Stella 2048

Evaluation: Throughput



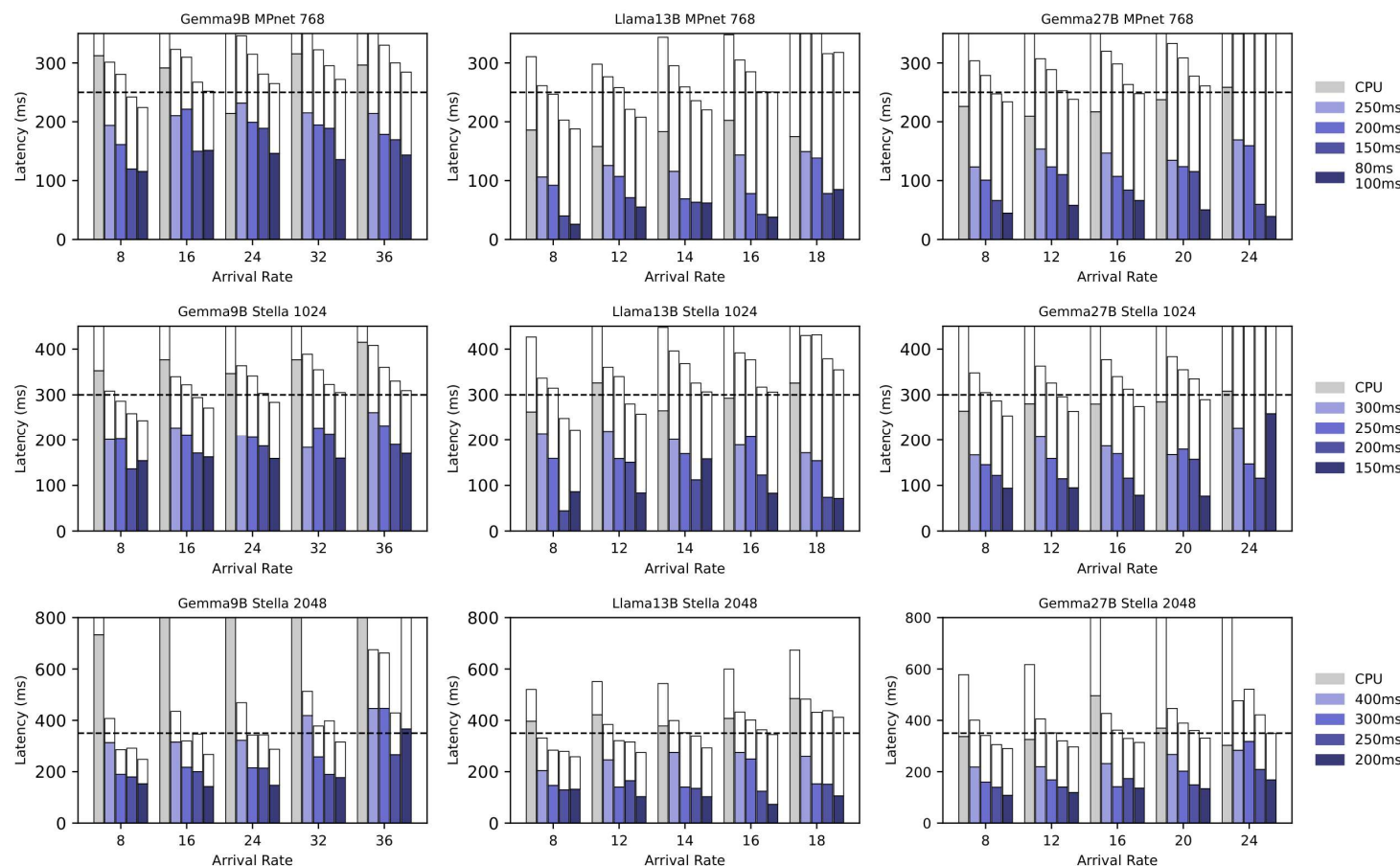
- Across different configurations, the estimation framework **effectively** predicts the performance of the partitioned index.
- While VectorLiteRag's predictions may deviate from observed values in certain cases, these deviations impact only performance estimation, not correctness.

Evaluation: Time-to-First Token



It improves performance, achieving a $3.1\times$ average speedup in TTFT reduction

Evaluation: Tail Latency



VectorLiteRag successfully meets the SLO target latency across various configurations.

Evaluation: Memory Demand

SLO_{search}	L40S	A100	H100
200ms	45 GB (56.3%)	23 GB (28.8%)	35 GB (43.8%)
250ms	35 GB (43.8%)	14 GB (17.5%)	27 GB (33.8%)
300ms	27 GB (33.8%)	12 GB (15.0%)	20 GB (25.0%)
400ms	23 GB (28.8%)	7 GB (8.8%)	12 GB (15.0%)

Table 1: Size of Partitioned Indexes of Stella 2048 Database

VectorLiteRag effectively determines the optimal partitioning point tailored to different system configurations, ensuring efficient resource utilization while maintaining performance.

Summary

• Problem

- In RAG systems, existing **vector search** and **LLM serving** optimizations are developed in **isolation**, leading to suboptimal end-to-end performance.
- Naively moving vector indices to GPU memory causes **resource contention**, reducing LLM's **KV cache** space and degrading generation performance.
- **Vector search** latency can be a bottleneck in RAG systems with large vector databases, and the serialized pipeline in RAG systems introduces latency overhead.

• Solution

- Adaptive Vector Index Partitioning
- Concurrent Search of Partitioned Index

• Result

- VectorLiteRAG improves vector search **responsiveness** by $2\times$.
- It significantly reduces end - to - end **TTFT** in RAG systems, achieving an average TTFT reduction of $2.2\times$ compared to the baseline and a $3.1\times$ speedup with larger vector databases.
- It successfully meets **the SLO target latency** across various configurations while traditional baselines violate latency requirements.